Apollo Data Stream API - Design

Overview

The **Apollo Data Stream API** provides endpoint that sends messages to requested Queues & Topics in Azure Service Bus and from there function apps send the messages to Pega.

> This is an externally hosted endpoint which Apollo can invoke.

Key Entities

The following are the core business entities related to streaming within the Domain API context.

- **Stream Message**
  - Entity related to the message to be streamed to Pega
  - Send message to the requested Topic, Queue in Service Bus

Technical Product Owner(s): Anbarasan Rajangam

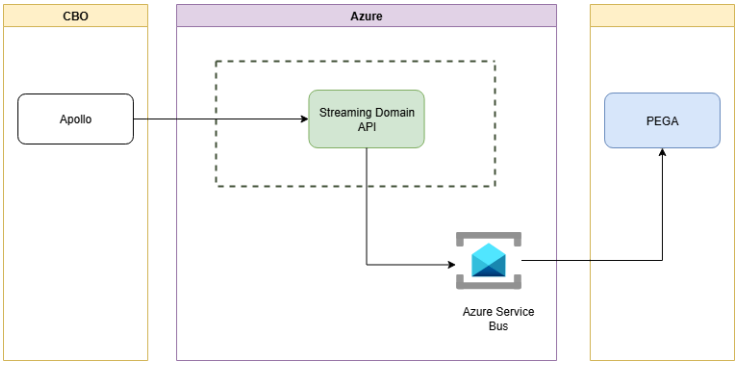
Stakeholders:

The key stakeholders are listed below, for the full list of project stakeholders refer [Project Stakeholders - DotNet Team - Confluence]

Upstream App name	Technical Owner
Apollo	Albert Gouzali / Andre Margono

High-Level Integration Diagram

Below is the high-level interaction diagram showing key upstream applications and other external/internal systems



Domain Model

Non-Restful Model

```
{
    "streamMessage": {
        "id": "string",
        "bupa_event_sequence": "string",
        "bupa_eventsource": "string",
        "bupa_eventtime": "datetime",
        "bupa_eventtype": "string",
        "bupa_primaryentityid": "string",
        "bupa_queuename": "string",
        "bupa_subscribername": "string",
        "bupa_topicname": "string",
        "body": "object"
    }
}
```

Upstream and Downstream Systems

- **Upstream systems:** The following systems shall access Streaming Domain API

- Apollo
- **Downstream systems:** The following systems shall be accessed by Streaming Domain API
  - Azure Service Bus
  - CRM

Caching

Caching Component Details

[Caching Design - DotNet Team - Confluence]

Integration Approach

How to integrate HI.Library.Cache component with domain API

Authentication

Authentication for the Membership Domain API is implemented using **Bearer Token Authentication** (OAuth 2.0 + JWT).

- **Client Applications** obtain an access token from **Microsoft Entra** using either:
  - **User Credentials** (Public Clients), or
  - **Managed Identity** (Confidential Clients).
- The access token **must** include the required permissions as **scopes** or **roles**.
- For every request to the Streaming Domain API, the client application must send the access token as a **Bearer token** in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

- The Bearer token is validated, and access to the requested endpoint is granted accordingly.

Additional References

- **Thick Client to Domain API (Cyclops to Domain API)**
  Thick Client to Domain API (Cyclops to Domain API)- Authentication and Authorization Flow
- **API to API Flow (BFF to Domain API)**
  [Authentication and Authorization Flow – DotNet Team - Confluence]

Token Validation Logic

The JWT is validated at the **API Management (APIM)** level for every request. Validation checks include:

- Valid Issuer
- Valid Audience
- Token Signature
- Token Lifetime

If any validation fails, the request is rejected with a **401 Unauthorized** response.

> Note: Similar token validation functionality exists within the `HI.Library.Authorization` package and can be enabled or disabled via configuration. Currently, this package's token validation is bypassed because tokens are validated at the APIM level.

Authorisation

Authorisation Overview

Authorisation is performed based on the **scopes** or **role claims** present in the JWT bearer token of each request. The `HI.Library.Authorization` package is utilized by the Streaming Domain API to implement the authorisation logic.

How is Authorisation Implemented?

- All applicable permissions such as `StreamingDomainAPI.Create`, `StreamingDomainAPI.Read`, etc., will be initialized for authorisation purposes.

- Endpoints requiring authorisation will be annotated with the `[AuthorizePermission]` custom attribute, specifying the required permission as an argument.
  Example:

```
[AuthorizePermission("StreamingDomainAPI.Read")]
```

  - This would typically be used on an HTTP GET method.
    Please refer to the complete list of permissions in the Apollo Data Stream API - Design | Per Endpoint Access Rules section.
  - When a request is made to an endpoint, the authorisation handlers validate that the required permissions (specified in the `[AuthorizePermission]` attribute) exist within the `scope` or `roles` claim of the JWT.
  - If the required claims are present and valid, access to the endpoint is granted. Otherwise, the request is rejected with a **401 Unauthorized** response.

Mapping of Users/Groups to Roles

> **Note:** This section applies to **Public Client Applications** only.

- For **Cyclops**, users will be added to the Azure Entra group:
  `RES-CLD-Cyclops-Hugo-dev-access`
  This group is associated with the App Registration during User Onboarding.
- Similarly, other public client applications (involving user interaction) can have their own groups, which can be assigned to their respective App Registrations.

Per-Endpoint Access Rules

- Each endpoint **must** use the `[AuthorizePermission]` custom attribute provided by the `HI.Library.Authorization` package.
- The following permissions apply to the Streaming API Endpoints (list to be provided in the detailed API design documentation).

Permission	Applicable HTTP Verb
`StreamingDomainAPI.Create`	`POST`

Logging

The **Logging component** provides a centralized logging solution for Hugo Domain APIs. It is implemented as a separate class library, enabling seamless integration across microservices as a reusable NuGet package.

- **Package Name:** `HI.Library.Logging`
- **Logging Design Documentation:**
  Logging - Design

Architecture & Implementation

- Logs are sent to **Azure Application Insights** for centralized monitoring and analysis.
- Uses the `ILogger` interface from **Microsoft.Extensions.Logging** to capture logs across Domain APIs.
- **OpenTelemetry** is leveraged to collect and export logs to Azure Application Insights via the `Azure.Monitor.OpenTelemetry.AspNetCore` package.

Key Logged Fields

Field Name	Description	Field Type
Timestamp	Date and time when the log entry was created.	Telemetry
Message	Main content of the log describing the event or issue.	Telemetry
Severity Level	Severity of the log (Info(1), Warning(2), Error(3), Critical(4)).	Telemetry
Operation ID	Unique identifier for the API request to correlate logs across services.	Telemetry
Operation Parent Id	ID of the parent operation to correlate nested operations (e.g., request & dependencies).	Telemetry

Cloud Role Name	Role name of the cloud service where telemetry data originated.	Telemetry
Method	HTTP method used in the request (GET, POST, PUT, DELETE).	Telemetry / Custom
Path	URL path of the request indicating the accessed endpoint.	Telemetry / Custom
Trace Id	Unique identifier for a single trace.	Telemetry
Item Type	Type of telemetry item (request, exception, trace).	Telemetry
Session ID	Identifier for a user session tracking user actions over a timeframe.	Custom
Customer ID	Unique identifier for a customer.	Custom
CorrelationID	Custom correlation ID to trace and correlate requests/responses across distributed services.	Custom

Log Levels and Retention

Log Level	Description
Information	Informational messages about normal application operation.
Warning	Potential issues or unexpected behaviors that do not disrupt application flow.
Error	Application errors or exceptions preventing specific operations from succeeding.
Critical	Severe errors causing application or critical component failures.

Retention Policy

- Application Insights retains log data for a configurable retention period (90 days).

Exception and Error Handling

Global Exception Handling Pattern

Exceptions are handled using the **Global Exception Handling Middleware** provided by the `HI.Library.ExceptionHandling` package.

Mapping Exception Types to HTTP Status Codes

Exception Type	HTTP Status Code	Description
Bad Request	400 Bad Request	The server could not understand the request due to invalid syntax.
Unauthorized	401 Unauthorized	Authentication is required and has failed or has not yet been provided.
Forbidden	403 Forbidden	The client does not have access rights to the content.
NotFound	404 Not Found	The server can’t find the requested resource.
RequestTimeout	408 Request Timeout	The server timed out waiting for the request.
Conflict	409 Conflict	The request conflicts with the current state of the server.
PreconditionFailed	412 Precondition Failed	The server does not meet one of the preconditions in the request headers.
Unprocessable Request	422 Unprocessable Entity	The request was well-formed but unable to be followed due to semantic errors.
InternalServerError	500 Internal Server Error	The server has encountered a situation it doesn't know how to handle.
NotImplemented	501 Not Implemented	The request method is not supported by the server and cannot be handled.
BadGateway	502 Bad Gateway	The server received an invalid response from the upstream server.

ServiceUnavailable	503 Service Unavailable	The server is not ready to handle the request, often due to maintenance.
GatewayTimeout	504 Gateway Timeout	The server did not receive a timely response from an upstream server.

Custom Error Schema

Error responses conform to the following JSON schema:

```
{
  "data": {},
  "items": [
    {
      "code": "string",
      "message": "string",
      "severity": "enum (Possible values: \"NotSet\", \"Information\", \"Warning\", \"Error\")",
      "classification": "enum (Possible values: \"Business\", \"System\")"
    }
  ]
}
```

Request Payload Storage in Application Insights

- When an exception occurs, the **request payload** is **encrypted** and stored in **Application Insights** as a property within the custom dimensions.
- Sensitive values are securely stored in **Azure Key Vault**.

Handling Custom Correlation Id (`X-Correlation-Request`)

- If a custom correlation ID is **not present** in the HTTP request header, the exception handling middleware **generates one** and assigns it to the header.
- If a correlation ID **already exists**, the middleware logs the exception in Application Insights using the **existing correlation ID**.

API Performance and Capacity Metrics

Metric Type	Metric	Value
Functional Load	Error rate [Failed requests percentage per minute]	0.01%
	Availability [System uptime percentage]	100%
Latency	Average response time	200 ms
	90th percentile	1 sec
	Worst case	1.5 sec
Throughput	Average requests per minute	600 calls/min
	Peak requests per minute	1500 calls/min
Expected Resource Use	Limits: Memory	1Gi
	Requests: CPU	100m
	Requests: Memory	750Mi
	HPA Configuration: Min Replicas	2
	HPA Configuration: Max Replicas	10

Infrastructure and Deployment Design

At the API level, define

- AKS namespace and container info:
  - Namespace: **hicbocoreservices-{env}-ns**
  - Container: **hi-api-streaming**
- Ingress/APIM setup

- APIM API Name - **histreamingdomain/{env}**
- ConfigMap and Secret usage
  - Library Variable Group: **hicbocoreservices-streaming-api-{env}-vars**
- Infrastructure CI/CD pipeline details:
  - **<To be Provided later>**
- Environment strategy (Dev, T27, Prod, etc.)
  - DEV: **d24**, SIT: **d34**, PERF: **t33**, UAT:**euat**, PROD: **prod**

Observability

AppDynamics/App Insights Integration Setup

- AppDynamics Tier/node mapping: NA
- AppDynamics Custom business transactions and metrics: NA
- App Insights: **hicbocoreservices-{env}-ai01**

AppDynamics/AppInsights Alerts

- CPU/memory/error rate thresholds: Api Alerts
- Notification channels: ServiceNow Ticket and Email Alert to dotnet support team.

Repository and Pipelines

Details	Link
Git repository	https://bupaaunz.visualstudio.com/HI-CBO-Apps/_git/HI.API.Streaming Connect your Azure Dev Ops account
Branching strategy	Branching Strategy for .NET based projects - DotNet Team - Confluence
Build and release pipelines	To be updated later
Security scans, tests, validations	To be updated later

Non-Functional Requirements

- Scalability: HPA should be enabled.
- Resilience and failover strategy (DR): API will be Active - Active running in both AKS clusters **productionaks** and **productiondraks.** For infrastructure DR refer the document:
  - Hugo Core Services and Batch Jobs Technical Recovery Procedure - Disaster Recovery
- Security and compliance standards (e.g., OWASP): SCR_008 API Security & Management
- Rate limiting and throttling: Rate limited on the API Level to **1000requests per IP/min**

Secrets Management

- Use of Key Vault or secret manager: Key Vaut will be used **banz-hi-streaming-{env}-kv01**
- Rotation and access control:
  - Manual yearly rotation of key vault entries has to be done for the following **<More details will be filled after the initial release>**

Key Vault Value	Description
ApimKeyValue	Subscription key for authenticating with Azure API Management (APIM).

API Specification

The below link has detailed design per end point

Streaming